



دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

عنوان:

واحد محاسبه با امکان انتخاب ثبات مبدا و مقصد

نام درس

آزمایشگاه معماری کامپیوتر

نیم سال ۱۴۰۴

استاد

دکتر سربازی آزاد

گروه ۲

امیرشایان سلیمانی نیا - ۴۰۳۱۰۶۱۰۸

محمد مهدی مرادی - ۴۰۳۱۰۶۶۸۱

فهرست مطالب

۱	مقدمه و هدف آزمایش	۲
۲	شرح آزمایش	۲
۳	ساختار و مراحل طراحی	۳
۱.۳	زیرمدار ALU	۳
۲.۳	زیرمدار Reg_File	۳
۳.۳	مدار Main	۴
۴	تست مدار	۶
۱.۴	مرحله اول	۶
۲.۴	مرحله دوم	۶
۳.۴	مرحله سوم	۷
۴.۴	مرحله چهارم	۷
۵	نتیجه گیری	۷

۱ مقدمه و هدف آزمایش

هدف این آزمایش، طراحی بخش‌های اولیه یک کامپیوتر ساده شامل ALU و مجموعه رجیسترها است تا در نهایت بتوانیم برنامه‌های زبان ماشین را روی آن اجرا کنیم.

۲ شرح آزمایش

در این آزمایش یک پردازنده ۸ بیتی پایه را در Logisim طراحی میکنیم که شامل ۴ رجیستر عمومی (R0 تا R3) و یک ALU با قابلیت انجام عملیات جمع و تفریق است. در این Datapath، یکی از عملوندها همواره همان رجیستر مقصد است و کنترل کل مسیر توسط یک دستور ۶ بیتی انجام می‌شود.

مدار طراحی شده شامل ورودی‌ها و خروجی‌های زیر است:

ورودی‌ها:

- دستور ۶ بیتی (Inst): این ورودی کنترل کل مدار را بر عهده دارد و از طریق یک Splitter به سه بخش تقسیم می‌شود:

- بیت ۵ (Op): تعیین‌کننده نوع عملیات (۰ برای جمع، ۱ برای تفریق).
- بیت‌های ۴ و ۳ (Dest): انتخاب رجیستر مقصد (یکی از رجیسترهای R0 تا R3) برای ورود به ALU و همچنین ذخیره نتیجه نهایی.
- بیت‌های ۲ تا ۰ (Source): انتخاب عملوند دوم با استفاده از یک Multiplexer، که می‌تواند محتوای یکی از رجیسترها یا مقادیر ثابت ۰، ۱ و ۱- باشد.

- سیگنال Clock: به دلیل ترتیبی بودن مدار از آن برای پایه کلاک در رجیسترها استفاده میشود.

- سیگنال Clr: این سیگنال در دستور کار وجود نداشت و فقط برای ساده تر شدن تست و ریست شدن مدار، به مدار اضافه شده است.

خروجی‌ها:

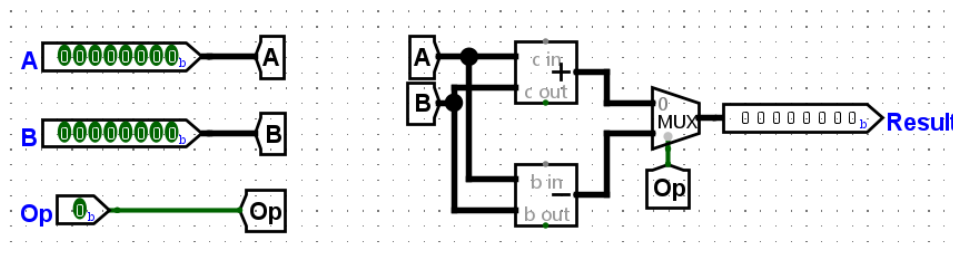
- محتوای رجیسترها (R0 تا R3): خروجی اصلی و نهایی این مدار، پین‌های ۸ بیتی متصل به رجیسترها هستند.

۳ ساختار و مراحل طراحی

مدار از دو زیر مدار و یک بخش اصلی تشکیل شده است در ادامه مراحل پیاده سازی هر بخش توضیح داده شده است:

۱.۳ زیرمدار ALU

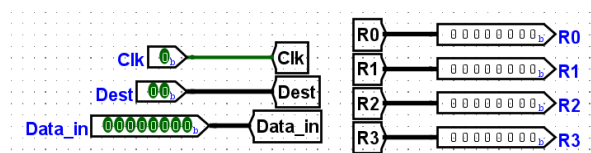
این بخش برای انجام محاسبات میباشد. از دو قطعه Adder و Subtractor هشت بیتی استفاده کردیم که ورودی های A و B به هر دوی آنها متصل شده اند. برای انتخاب بین نتیجه جمع یا تفریق، خروجی این دو قطعه را به یک Multiplexer وصل کردیم که توسط پین کنترلی Op مدیریت می شود؛ در صورت یک بودن سیگنال Op، باید عملیات تفریق انجام شود و در غیر این صورت، حاصل عملیات جمع به خروجی می رود.



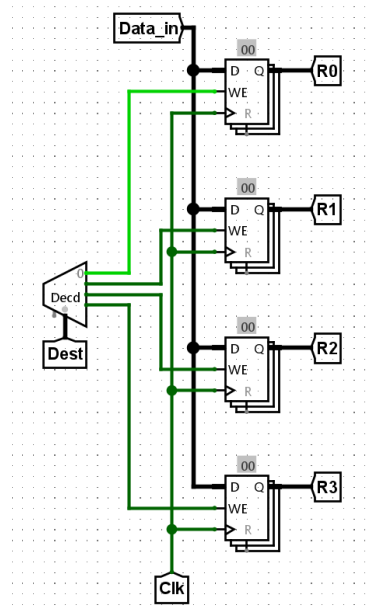
شکل ۱: زیرمدار ALU برای عملیات جمع و تفریق

۲.۳ زیرمدار Reg_File

این زیرمدار وظیفه نگهداری داده ها را بر عهده دارد. از چهار قطعه Register هشت بیتی استفاده کردیم. Data_in به پایه ی داده ی تمام رجیسترها متصل است. برای اینکه در هر سیکل فقط رجیستر هدف به روزرسانی شود، از یک Decoder استفاده کردیم که ورودی Dest را دریافت کرده و فقط پایه Enable رجیستر مورد نظر را فعال می کند. سیگنال Clock نیز به همه رجیسترها متصل شده است.



شکل ۲: ورودی ها و خروجی ها



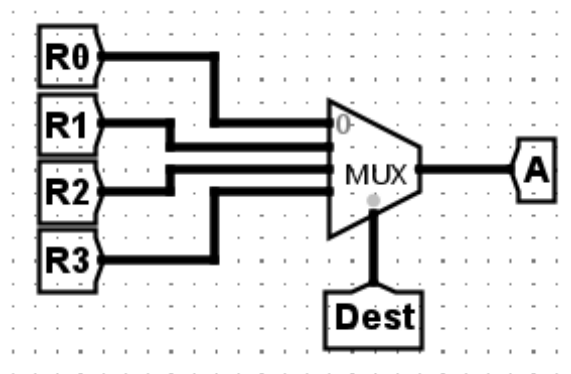
شکل ۳: انتخاب رجیسترها با دیکدر

۳.۳ مدار Main

مدار اصلی از به هم پیوستن زیرمدارها و ایجاد Datapath تشکیل شده است. این مدار را در سه بخش مجزا بررسی می‌کنیم:

بخش اول: انتخاب عملوند اول

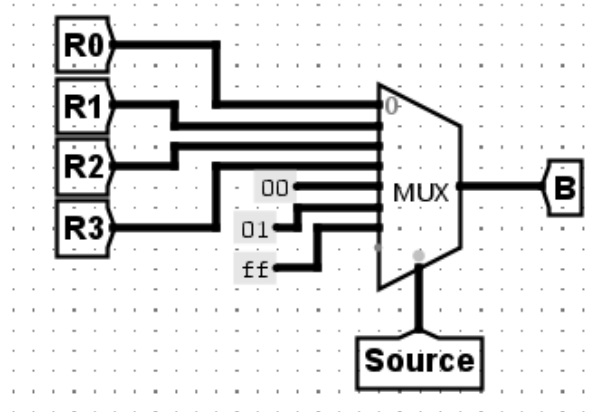
برای انتخاب عملوند اول (A) که قرار است وارد ALU شود، از یک Multiplexer استفاده کردیم. ورودی‌های این مالتی‌پلکسر، خروجی رجیسترها هستند و سیگنال انتخاب‌گر آن، پین Dest است تا همیشه محتوای فعلی رجیستر مقصد به عنوان عملوند اول وارد محاسبه شود.



شکل ۴: مسیر انتخاب A

بخش دوم: انتخاب عملوند دوم

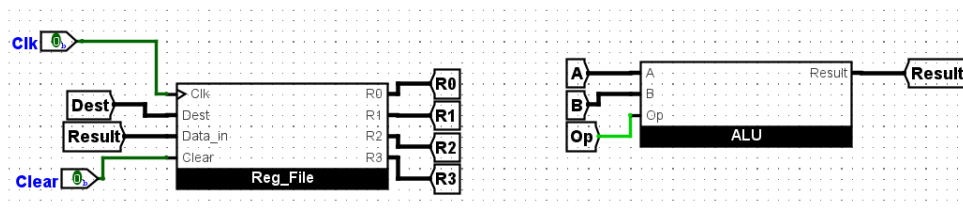
عملوند دوم (B) می‌تواند حالات مختلفی داشته باشد. برای این مسیر از یک Multiplexer بزرگ‌تر استفاده کردیم. سیگنال Source تعیین می‌کند که مقدار ورودی دوم به ALU، محتوای یکی از رجیسترها باشد یا یکی از مقادیر ثابت ۰ (0x00)، ۱ (0x01) و ۰xFF (0xFF).



شکل ۵: مسیر انتخاب B

بخش سوم: پردازش نتیجه

عمل اصلی پردازش در این بخش و توسط دو زیرمدار ALU و Reg_File انجام می‌شود. عملوندهای A و B به همراه سیگنال Op (برای تعیین عملیات جمع یا تفریق) وارد واحد ALU می‌شوند. خروجی نهایی (Result) به Reg_File بازگردانده می‌شود.



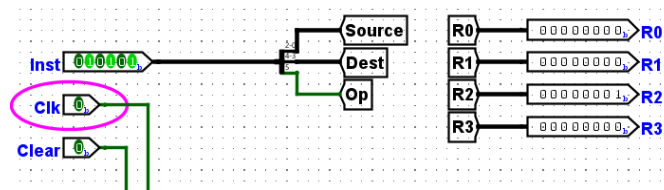
شکل ۶: اتصال زیرمدارها و تکمیل حلقه Datapath

۴ تست مدار

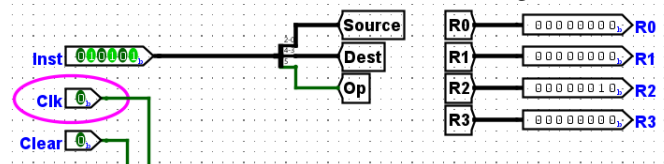
برای تست مدار، ۴ مرحله در نظر میگیریم که به ترتیب اجرا، آن ها را توضیح میدهیم:

۱.۴ مرحله اول

در این مرحله هدف ما جمع کردن مقدار ثابت ۱ با رجیستر R2 است دستور ۶ بیتی 010101 را در ورودی Inst وارد می‌کنیم که به معنای انتخاب عملیات جمع، مقصد R2 و مبدأ ثابت ۱ است. برای این مرحله دو پالس Clock اعمال می‌کنیم تا مقدار ۱ دو بار با رجیستر جمع شود. همان‌طور که در تصاویر مشخص است، پس از کلاک اول مقدار R2 برابر با ۱ و پس از کلاک دوم برابر با ۲ (00000010) می‌شود.



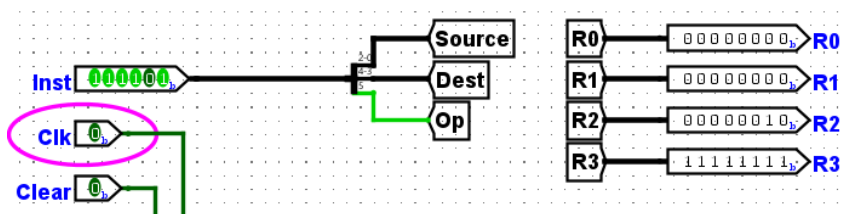
شکل ۷: مرحله اول - نتیجه پس از کلاک اول



شکل ۸: مرحله اول - نتیجه پس از کلاک دوم

۲.۴ مرحله دوم

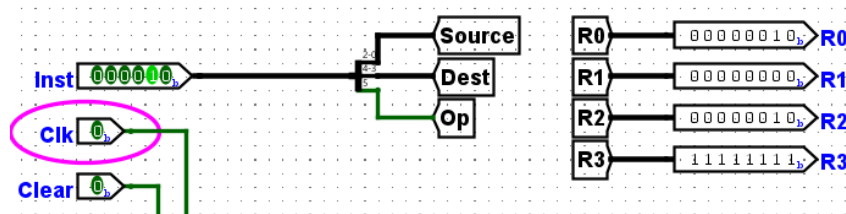
در این مرحله می‌خواهیم عدد ثابت ۱ را از رجیستر R3 که مقدار اولیه آن صفر است کم کنیم. دستور 111101 را به ورودی می‌دهیم (Op=1 برای تعیین عملیات تفریق) با اعمال یک پالس Clock، مقدار R3 از صفر به منفی یک (11111111) تغییر می‌کند.



شکل ۹: مرحله دوم - تفریق ثابت ۱ و تولید عدد منفی در R3

۳.۴ مرحله سوم

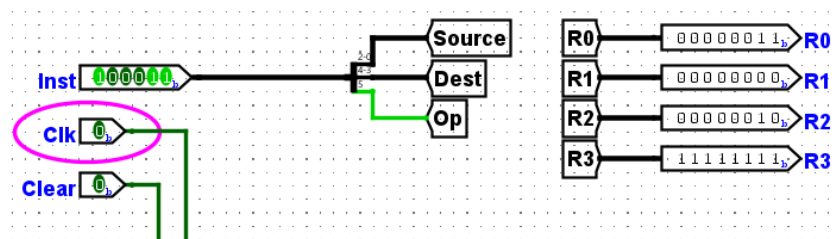
اکنون قصد داریم مقدار فعلی R2 (که برابر با ۲ است) را با R0 (که مقدارش صفر است) جمع کنیم برای این کار دستور 000010 را وارد می‌کنیم. با زدن یک پالس Clock، مقدار رجیستر R2 با موفقیت در رجیستر R0 جمع می‌شود و خروجی R0 نیز برابر با ۲ (00000010) می‌شود.



شکل ۱۰: مرحله سوم - انتقال و جمع محتوای R2 با R0

۴.۴ مرحله چهارم

در مرحله نهایی، می‌خواهیم محتوای R3 (که منفی یک است) را از R0 (که در مرحله قبل ۲ شد) تفریق کنیم دستور 100011 را اعمال کرده و یک پالس Clock می‌زنیم. مقدار 00000011 را در رجیستر R0 ذخیره و نمایش داده می‌شود.



شکل ۱۱: مرحله چهارم - تفریق محتوای R3 از R0

۵ نتیجه‌گیری

در این آزمایش، ما توانستیم معماری پایه یک کامپیوتر ساده شامل واحد محاسبات و مجموعه رجیسترهای ۸ بیتی را طراحی و پیاده‌سازی کنیم. با تقسیم کردن مدار به بلوک‌های مختلف مثل ALU و Reg_File، توانستیم کارها را به بخش‌های کوچک‌تر و قابل مدیریت تقسیم کنیم. مدار ما بر اساس دستورات ۶ بیتی (Inst) عملیات مورد نظر را انتخاب می‌کند و با اعمال پالس سیگنال Clock، نتایج را به درستی در رجیستر مقصد ذخیره و به‌روزرسانی می‌کند.